

Module : Kotlin (M202) TP N° 14

Année de Formation 2024/2025

Filière : Développement Mobile Groupe : DEVM 101 - DEVM 102 Niveau : 2ème année

Tuto Coroutine dans IntelliJ Idea :

<https://www.youtube.com/watch?v=LIBoYfQIElg>

Exercice 1 :

- Crée une application Kotlin simple avec une fonction main.
- Dans cette application, tu vas simuler des opérations longues en utilisant des coroutines.
- Tu vas créer **3 types** de scopes :

GlobalScope : Lancer une coroutine avec GlobalScope.

CoroutineScope : Créer un scope personnalisé et gérer ses coroutines.

runBlocking : Bloquer le thread principal jusqu'à ce que toutes les coroutines soient terminées.

- Simule deux tâches longues (par exemple, téléchargement de fichier et traitement de données) en utilisant des délais (delay()).
- Expérimente avec l'annulation de coroutines en appelant cancel() dans le bon scope.
- Utilise les scopes pour observer comment cela affecte le comportement et le temps d'exécution des tâches.

Exercice 2 :

- Créer une application simple en Kotlin avec une fonction main.

- Utilise la fonction **launch** pour démarrer plusieurs coroutines simultanées.
- Simule des tâches longues (par exemple, télécharger des fichiers ou calculer des valeurs) en utilisant la fonction **delay()**.
- Utilise différents Dispatchers pour exécuter les coroutines :

Dispatchers.Default pour des tâches qui nécessitent du calcul.

Dispatchers.IO pour des tâches liées aux entrées/sorties (comme lire des fichiers ou faire des appels réseau).

Dispatchers.Main (si tu travailles dans une application Android) pour exécuter sur le thread principal.

- Utilise **runBlocking** pour attendre la fin de toutes les coroutines.

Exercice 3 :

- Crée une application simple en Kotlin avec une fonction **main**.
- Utilise la fonction **async** pour lancer plusieurs coroutines asynchrones qui retournent des valeurs (par exemple, des calculs ou des appels réseau simulés avec **delay()**).
- Utilise la fonction **await** pour attendre et obtenir les résultats des coroutines.
- Expérimente avec l'exécution en parallèle de plusieurs tâches asynchrones pour voir comment elles se comportent lorsqu'on attend les résultats.

Exercice 4 :

- Créer une application simple en Kotlin avec une fonction **main**.
- Lance plusieurs coroutines à l'aide de la fonction **launch** et assigne-les à des objets **Job**.
- Utilise les fonctions suivantes sur les **Jobs** :

isActive : Pour vérifier si un job est toujours actif.

cancel() : Pour annuler un job.

join() : Pour attendre qu'un job se termine.

- Expérimente avec l'annulation de certaines coroutines pour voir comment cela affecte leur exécution.

Exercice 5 :

- Crée un **Flow** qui émet une série de nombres de 1 à 10.
- Utilise différents opérateurs de débit pour transformer et filtrer les données émises.
- Collecte les résultats et affiche-les à la console.
- Expérimente avec les opérateurs suivants :

map : pour transformer les valeurs.

filter : pour filtrer les valeurs.

take : pour ne prendre qu'un nombre limité d'éléments.

collect : pour collecter et afficher les données du flux.

Exercice 6 :

- Créer une application Kotlin avec une fonction **main**.
- Créer un **CoroutineScope** personnalisé avec un Job et un Dispatcher.
- Lance plusieurs coroutines à l'intérieur de ce scope pour simuler des tâches asynchrones.
- Utilise les fonctions de **CoroutineScope** comme **launch**, **cancel**, et **join**.
- Expérimente en annulant le scope pour observer comment cela affecte les coroutines en cours d'exécution.

